

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>

#define MAX_ARGS 64
#define BUF_SIZE 1024

// Function pointer definition for built-in command handlers
typedef int (*BuiltinHandler)(char **args);

typedef struct {
    const char *name;
    BuiltinHandler handler;
} BuiltinCommand;

// Forward declaration of built-in handlers
int builtin_echo(char **args);
int builtin_setenv(char **args);
int builtin_help(char **args);

// Dispatch Table for In-Process Commands
BuiltinCommand dispatch_table[] = {
    {"echo", builtin_echo},
    {"setenv", builtin_setenv},
    {"help", builtin_help},
    {NULL, NULL}
};

// Built-in Handler Implementations
int builtin_echo(char **args) {
    for (int i = 1; args[i] != NULL; i++) {
        printf("%s%s", args[i], args[i + 1] ? " " : "");
    }
    printf("\n");
    return 0;
}
```

```

}

int builtin_setenv(char **args) {
    if (args[1] == NULL || args[2] == NULL) {
        printf("Usage: setenv <VAR> <VALUE>\n");
        return 1;
    }
    if (setenv(args[1], args[2], 1) == 0) {
        printf("Variable '%s' set to '%s'\n", args[1], args[2]);
    } else {
        perror("setenv failed");
    }
    return 0;
}

int builtin_help(char **args) {
    printf("--- Shell Built-in Commands ---\n");
    printf("1. echo <text>          - Print arguments\n");
    printf("2. setenv <var> <val>    - Set an environment variable\n");
    printf("3. help                  - Display this menu\n");
    printf("4. exit                  - Exit the shell\n");
    return 0;
}

// 1. VARIABLE EXPANSION ENGINE
char* expand_variables(const char *input) {
    char *result = malloc(BUF_SIZE);
    int res_idx = 0;
    int len = strlen(input);
    int i = 0;

    while (i < len) {
        // Detect variable reference '$'
        if (input[i] == '$') {
            i++;
            char var_name[128];
            int var_idx = 0;

```

```

        // Support nested/adjacent variables by reading variable name characters
        while (i < len && (isalnum(input[i]) || input[i] == '_')) {
            var_name[var_idx++] = input[i++];
        }
        var_name[var_idx] = '\0';

        if (var_idx > 0) {
            // Lookup variable; handle undefined variables safely
            char *val = getenv(var_name);
            if (val) {
                while (*val && res_idx < BUF_SIZE - 1) {
                    result[res_idx++] = *val++;
                }
                // Undefined variables expand to nothing (empty string)
            } else {
                result[res_idx++] = '$'; // Isolated '$' symbol
            }
        } else {
            result[res_idx++] = input[i++];
        }
    }
    result[res_idx] = '\0';
    return result;
}

// 2. DISPATCH TABLE EXECUTION
int execute_builtin(char **args) {
    if (args[0] == NULL) return 0;

    for (int i = 0; dispatch_table[i].name != NULL; i++) {
        if (strcmp(args[0], dispatch_table[i].name) == 0) {
            // Execute in-process handler
            return dispatch_table[i].handler(args);
        }
    }
}

```

```

    return -1; // Not a built-in command
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nDispatch Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        if (strcmp(input_buf, "exit") == 0) break;

        // Step A: Expand Variable References
        char *expanded_input = expand_variables(input_buf);
        printf("[Debug Expansion] Original: \"%s\" -> Expanded: \"%s\"\n", input_buf, expanded_input);

        // Step B: Tokenize Input
        char *args[MAX_ARGS];
        int arg_count = 0;
        char *token = strtok(expanded_input, " ");

        while (token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " ");
        }
        args[arg_count] = NULL;

        // Step C: Dispatch In-Process Built-ins or handle external fallback
        if (arg_count > 0) {
            int result = execute_builtin(args);
            if (result == -1) {
                printf("[Dispatch] '%s' is an external/invalid command (not in dispatch table).\n", args[0]);
            }
        }
    }
}

```

```

        free(expanded_input);
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano vars_dispatch.c
abhis@MIKHAIL-37:~$ gcc -g -Wall vars_dispatch.c -o vars_dispatch
abhis@MIKHAIL-37:~$ ./vars_dispatch

Dispatch Shell > echo User: $USER Home: $HOME
[Debug Expansion] Original: "echo User: $USER Home: $HOME" -> Expanded: "echo User: abhis Home: /home/abhis"
User: abhis Home: /home/abhis

Dispatch Shell > echo Pre_${UNDEFINED_VAR_NAME}_Post $USER
[Debug Expansion] Original: "echo Pre_${UNDEFINED_VAR_NAME}_Post $USER" -> Expanded: "echo Pre_ abhis"
Pre_ abhis

Dispatch Shell > setenv MY_ROLE Developer
[Debug Expansion] Original: "setenv MY_ROLE Developer" -> Expanded: "setenv MY_ROLE Developer"
Variable 'MY_ROLE' set to 'Developer'

Dispatch Shell > help
[Debug Expansion] Original: "help" -> Expanded: "help"
--- Shell Built-in Commands ---
1. echo <text>          - Print arguments
2. setenv <var> <val>   - Set an environment variable
3. help                - Display this menu
4. exit                - Exit the shell

Dispatch Shell > custom_unknown_cmd
[Debug Expansion] Original: "custom_unknown_cmd" -> Expanded: "custom_unknown_cmd"
[Dispatch] 'custom_unknown_cmd' is an external/invalid command (not in dispatch table).
```

```
Dispatch Shell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./vars_dispatch
==1230== Memcheck, a memory error detector
==1230== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1230== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1230== Command: ./vars_dispatch
==1230==

Dispatch Shell > exit
==1230==
==1230== HEAP SUMMARY:
==1230==    in use at exit: 0 bytes in 0 blocks
==1230==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1230==
==1230== All heap blocks were freed -- no leaks are possible
==1230==
==1230== For lists of detected and suppressed errors, rerun with: -s
==1230== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```